

A Reproducible Benchmark for OCR Post-Correction of Low-Resource Arabic-Script Turkic Historical Texts

Radmir

Spring 2026

Abstract

This project studies OCR post-correction for historical Turkic texts written in Arabic script. We formulate the problem as a sequence-to-sequence NLP task: given a noisy OCR-like text line, the system should generate its clean version. We collect a small Arabic-script Turkic corpus, construct a synthetic noisy-clean benchmark, compare identity and rule-based baselines with a ByT5-small model, and evaluate the results with Character Error Rate (CER), Word Error Rate (WER), and Exact Match. On the proposed benchmark, ByT5-small with a 512-token sequence length achieves the best CER and WER among the evaluated methods. The project code is available at: <https://github.com/Therad445/low-resource-arabic-script-turkic-ocr>.

1 Introduction

Historical Turkic texts written in Arabic script are important for linguistic, cultural and historical research. However, such materials remain difficult to process automatically. The task is low-resource: there are few annotated corpora, orthography is unstable, document quality varies, and standard OCR and NLP tools are usually designed for modern high-resource languages and scripts.

This project focuses on OCR post-correction. We do not attempt to solve full image-based OCR or handwritten text recognition. Instead, we isolate the next NLP stage: correcting an already produced noisy text output. The task is formulated as:

noisy Arabic-script Turkic text $\rightarrow$ clean Arabic-script Turkic text.

The main contributions are:

- a small reproducible benchmark for Arabic-script Turkic OCR post-correction;
- a synthetic OCR-like noise generation protocol;

- a clean-line-level train/validation/test split without leakage;
- comparison of identity, rule-based and neural sequence-to-sequence approaches;
- evaluation with CER, WER and Exact Match;
- per-sample error analysis of model improvements and degradations.

1.1 Team

Radmir collected and prepared the corpus, constructed the synthetic post-correction dataset, implemented and evaluated the baselines, trained and evaluated the ByT5-small model, performed error analysis, prepared the repository and wrote the final report.

2 Related Work

2.1 Historical OCR and HTR

OCR for historical documents differs from OCR for modern clean printed text. Historical material often contains unstable typography, damaged scans, non-standard spelling, complex layouts and domain-specific writing conventions. OCR workflows for historical printings therefore usually include data preparation, ground-truth construction, model training, post-processing and manual validation [Reul et al., 2019, Springmann et al., 2018].

2.2 Arabic-Script OCR

Arabic-script OCR is challenging because letters change shape depending on their position, writing is cursive, and many characters differ only by small dots or diacritics. These issues become even more difficult in historical or multilingual contexts, where Arabic, Persian and Turkic orthographic traditions may overlap. Surveys of Arabic OCR emphasize that such script-specific properties require specialized handling [Naz et al., 2014].

2.3 OCR Post-Correction

OCR post-correction aims to transform noisy OCR output into a cleaner textual representation. At the line level, this can naturally be formulated as a sequence-to-sequence task. The quality of post-correction is commonly evaluated with edit-distance-based metrics such as CER and WER, which are based on insertions, deletions and substitutions [Levenshtein, 1966].

2.4 Transformer and Byte-Level Models

Transformer models [Vaswani et al., 2017] and text-to-text transfer learning [Raffel et al., 2020] provide a general framework for sequence-to-sequence NLP tasks. In this project, we use ByT5 [Xue et al., 2022], a byte-level sequence-to-sequence model. Byte-level modeling is useful for noisy and low-resource settings because it avoids dependence on a fixed word or subword vocabulary. This is especially relevant for historical Arabic-script Turkic text, where rare spellings, mixed orthographic conventions and OCR-like distortions can make subword tokenization fragile.

3 Model Description

The project compares three approaches.

3.1 Identity Baseline

The identity baseline returns the noisy input unchanged. It measures the initial quality of the synthetic OCR-like text and serves as the simplest reference point.

3.2 Rule-Based Normalizer

The rule-based normalizer applies simple Arabic-script normalization rules. This baseline tests whether naive character normalization is useful for this material. Since historical orthography may contain meaningful variation, aggressive normalization can be harmful.

3.3 ByT5-small

The main neural approach is ByT5-small. It is fine-tuned as a sequence-to-sequence model. The input is a noisy OCR-like line, and the target is the corresponding clean line. The final evaluated configuration uses 512-token source and target sequence lengths and is trained for 2 epochs.

The use of a byte-level model is motivated by the low-resource and noisy nature of the task. Arabic-script characters may occupy multiple bytes, and historical spelling variation can be poorly represented by standard subword tokenizers. Therefore, sequence length is an important experimental parameter.

4 Dataset

The dataset is a pilot benchmark for OCR post-correction of Arabic-script Turkic historical text. The clean corpus was collected from an Ottoman Turkish / Arabic-script Turkic Wikisource source. Synthetic OCR-like errors were then generated to create noisy-clean pairs.

Each processed CSV file contains:

- `line_id`: identifier of the original clean line;
- `variant_id`: identifier of the noisy variant;
- `noisy`: synthetically corrupted OCR-like text;
- `clean`: original clean text.

The split is performed by clean source lines. This prevents leakage: noisy variants of the same original line do not appear in different splits. The source description is stored in `data/postcorrection/raw/SOURCES.md`. This makes the benchmark suitable for controlled comparison of post-correction methods, although it does not replace evaluation on real OCR or HTR output.

Split	Clean lines	Noisy-clean pairs	Avg. clean chars	Avg. noisy chars
train	320	6400	113.94	113.51
valid	40	800	112.75	112.24
test	40	800	122.25	121.70

Table 1: Dataset split statistics for the proposed Arabic-script Turkic OCR post-correction benchmark.

5 Experiments

5.1 Metrics

We use three metrics: Character Error Rate (CER), Word Error Rate (WER), and Exact Match.

CER is computed as:

$$CER = \frac{S + D + I}{N},$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the number of characters in the reference text.

WER is computed analogously at the word level:

$$WER = \frac{S_w + D_w + I_w}{N_w}.$$

Exact Match is the fraction of examples where the predicted line exactly matches the clean reference line.

5.2 Experiment Setup

The benchmark contains train, validation and test splits. The final evaluation is performed on the test split. The identity and rule-based baselines are evaluated directly on the test set. The ByT5-small model is fine-tuned on the training split and evaluated on the test split.

The final ByT5-small configuration uses 512-token source and target sequence lengths and 2 training epochs. The main results are stored in `outputs/postcorrection/final_metrics.csv`.

5.3 Baselines

The project includes two baselines. The identity baseline leaves noisy text unchanged. The rule-based normalizer applies simple Arabic-script normalization rules. These baselines are necessary because there is no previous published result on the proposed benchmark.

6 Results

Method	CER ↓	WER ↓	Exact Match ↑	N
Identity baseline	0.086005	0.519006	0.001250	800
Rule-based normalizer	0.151932	0.684408	0.000000	800
ByT5-small 512 / 2 epochs	0.079913	0.368540	0.003750	800

Table 2: Main test results on the proposed benchmark. Lower CER and WER are better; higher Exact Match is better.

The ByT5-small 512-token model achieves the best CER and WER among the evaluated methods on the proposed benchmark. The rule-based normalizer performs worse than the identity baseline. This suggests that naive normalization can distort meaningful orthographic properties of historical Arabic-script Turkic text.

The improvement is stronger for WER than for CER. This indicates that the model often repairs word-level structure even when exact character-level recovery remains difficult. Exact Match remains low for all methods because the lines are long and orthographically complex.

6.1 Error Analysis

The error analysis shows that the model improves most test examples, but it is not uniformly safe. Some predictions are worse than the noisy input. Therefore, the model should be treated as a useful post-correction component rather than a fully automatic scholarly transcription tool.

Criterion	Improved	Unchanged	Worse	Total
CER	682	60	58	800
WER	677	63	60	800

Table 3: Per-sample error analysis for ByT5-small 512 compared with the noisy input.

7 Limitations

The current benchmark is intentionally limited. First, the corpus is small and comes from one main source. Second, the noisy-clean pairs are based on synthetic OCR-like corruptions rather than real OCR or HTR outputs. Third, the model is evaluated automatically and has not yet been validated by a domain expert. Finally, some ByT5 predictions are worse than the original noisy input, so the model should be used as an assistive post-correction component rather than a fully automatic scholarly transcription system.

8 Conclusion

This project introduced a reproducible pilot benchmark for OCR post-correction of low-resource Arabic-script Turkic historical text. We collected a small clean corpus, generated synthetic OCR-like noisy-clean pairs, created a train/validation/test split without clean-line leakage, and compared identity, rule-based and ByT5-small approaches.

The ByT5-small 512-token configuration achieved the best CER and WER among the evaluated methods on the proposed benchmark. At the same time, the experiment has important limitations: the corpus is small, the current data comes from one main source, errors are synthetic, and real OCR/HTR outputs should be added in future work. The project therefore represents a reproducible pilot study and a foundation for future work rather than a finished OCR system.

References

- [Levenshtein, 1966] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710.
- [Naz et al., 2014] Naz, S., Umar, A. I., Ahmad, R., Siddiqi, I., Ahmed, S. H., Razzak, M. I., and Shafait, F. (2014). A survey of ocr in arabic language: Applications, techniques, and challenges. *Applied Sciences*, 4(3):397–420.
- [Raffel et al., 2020] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.

- [Reul et al., 2019] Reul, C., Christ, D., Hartelt, A., Balbach, N., Wehner, M., Springmann, U., Wick, C., Grundig, C., Büttner, A., and Puppe, F. (2019). Ocr4all—an open-source tool providing a semi-automatic ocr workflow for historical printings. *Applied Sciences*, 9(22):4853.
- [Springmann et al., 2018] Springmann, U., Fink, F., and Schulz, K. U. (2018). Ocr of historical printings with an application to building diachronic corpora: A case study using the ridges herbal corpus. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation*.
- [Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*.
- [Xue et al., 2022] Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., and Raffel, C. (2022). Byt5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306.